Structure cleanup: the 5 rules

`optimize_prog_structure` in `optimizer_simple_egglog.py`, lines 917–947 (branch dead-code-elimination). Every example below was run through the real function.

What it does

A purely syntactic pass over the Prog tree. It works bottom-up: the body of every **Cond** and every element of a **Sequence** is cleaned first, then the rules are applied to the cleaned result. It looks at no variable values. The pipeline runs it three times (after the first dead-code pass, after the ite lowering, after the second dead-code pass, lines 1939, 1941, 1943) and once per round inside the dead-code loop (line 1909).

The rules

#	Rule	Before	After	Why it is safe

1	A guard that is the constant **true** is replaced by its body	`if (true) {`
		`  y = 1;`
		`}`
			`y = 1;`	The branch is always taken.

2	A guard that is the constant **false** removes the whole **if**	`if (false) {`
		`  y = 1;`
		`}`
			(nothing)	The branch is never taken.

3	An **if** whose (cleaned) body is empty is removed	`if (x < 0) {` `}`	(nothing)	Evaluating a guard has no effect: terms are side-effect free.

4	A block inside a block is **flattened**	`{ a = 1;`
		`  { b = 2; c = 3; }`
		`  d = 4; }`
			`{ a = 1;`
			`  b = 2; c = 3;`
			`  d = 4; }`
				Sequencing is associative.

5	An **empty block** inside a block is dropped	`{ a = 1;`
		`  { }`
		`  b = 2; }`
			`{ a = 1;`
			`  b = 2; }`
				The empty block does nothing (it is the unit of sequencing).

Rule 5 is the same effect as rule 4: flattening an empty block already adds nothing to the list. The extra filter that drops empty blocks in the code never changed a result in 20,000 random programs.

Worked example (the innermost part is cleaned first)

The inner guard becomes empty, so the guard above it becomes empty, and so on up the tree. The result is still a block with one statement: a one-element block is not unwrapped.

Step	Rule	Program becomes
start		`if (true) { if (x < 0) { if (false) { y = 1; } } z = 2; }`
1	2	`if (true) { if (x < 0) { } z = 2; }`
2	3	`if (true) { { } z = 2; }`
3	4 / 5	`if (true) { z = 2; }`
4	1	`{ z = 2; }`

What it leaves alone

- **Empty infinite loops** stay: `loop { }` never reads and never ends, so removing it would change the behaviour.
- **One-element blocks** are not unwrapped, and an assignment with an empty list is not removed.
- **Non-constant guards** are never decided here. Terms like `x < x` are simplified by Egglog (rule #75), and guards decided by known values are handled by the dead-code rule R2.

Checks and meaning

- Running the pass twice gives the same result as running it once (5,000 random programs).
- All five rules change only **internal steps**, never what the environment sees at a Read and never how the program ends: in the model of `Prog.v` they are equivalences up to internal steps (weak bisimulation). That statement is argued here, **not yet proved in Coq**; the Coq proof so far covers only the Egglog stage.